

Création d'une application de Machine Learning pour la détection des faux billets

Gwenaëlle Boussès
Data Analyst

Contexte et objectif



ONCFM = ORGANISATION PUBLIQUE



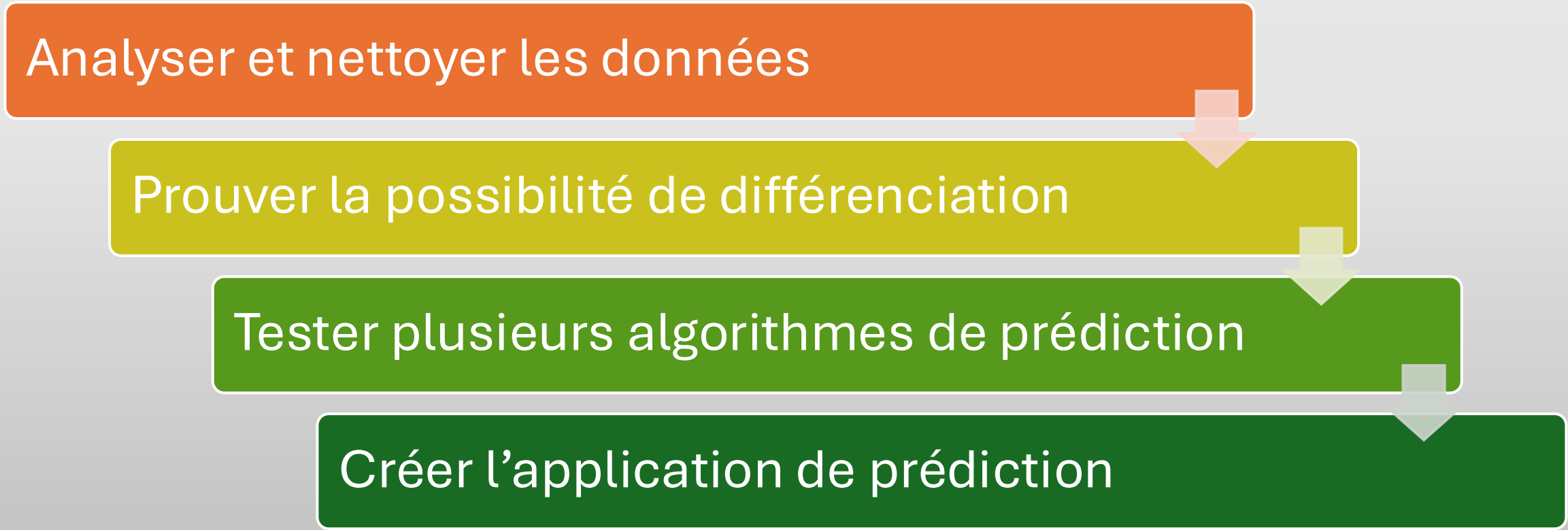
OBJECTIF : METTRE EN PLACE DES MÉTHODES
D'IDENTIFICATION DES FAUX BILLETS



MISSION : CRÉATION D'UN ALGORITHME
CAPABLE DE DÉFINIR LA VÉRACITÉ D'UN BILLET
À PARTIR DE SES CARACTÉRISTIQUES
GÉOMÉTRIQUES

Démarche globale

Analyser et nettoyer les données



```
graph TD; A[Analyser et nettoyer les données] --> B[Prouver la possibilité de différenciation]; B --> C[Tester plusieurs algorithmes de prédiction]; C --> D[Créer l'application de prédiction];
```

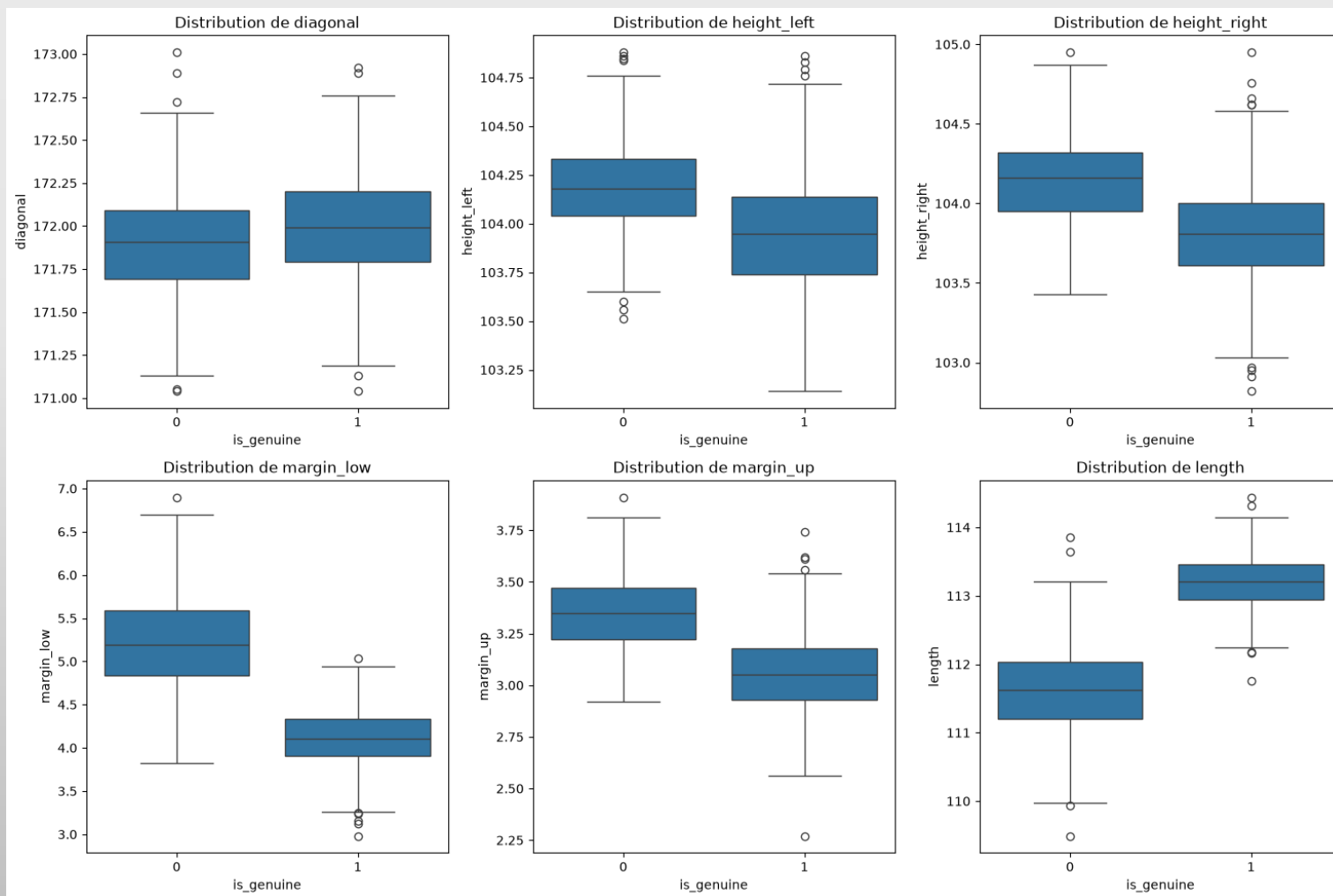
The diagram illustrates a four-step global approach. It consists of four horizontal rectangular boxes arranged in a descending staircase pattern from top-left to bottom-right. The first box is orange, the second is yellow-green, the third is green, and the fourth is dark green. Each box contains a step in the process. Downward-pointing arrows connect the bottom-right corner of one box to the top-right corner of the next box below it, indicating a sequential flow.

Prouver la possibilité de différenciation

Tester plusieurs algorithmes de prédiction

Créer l'application de prédiction

Analyse des données

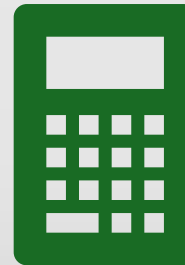


Données manquantes : que faire ?



Remplacement des données manquantes par la médiane

- N'est pas sensible aux outliers (contrairement à la moyenne)
- représente strictement la valeur du milieu : plus robuste

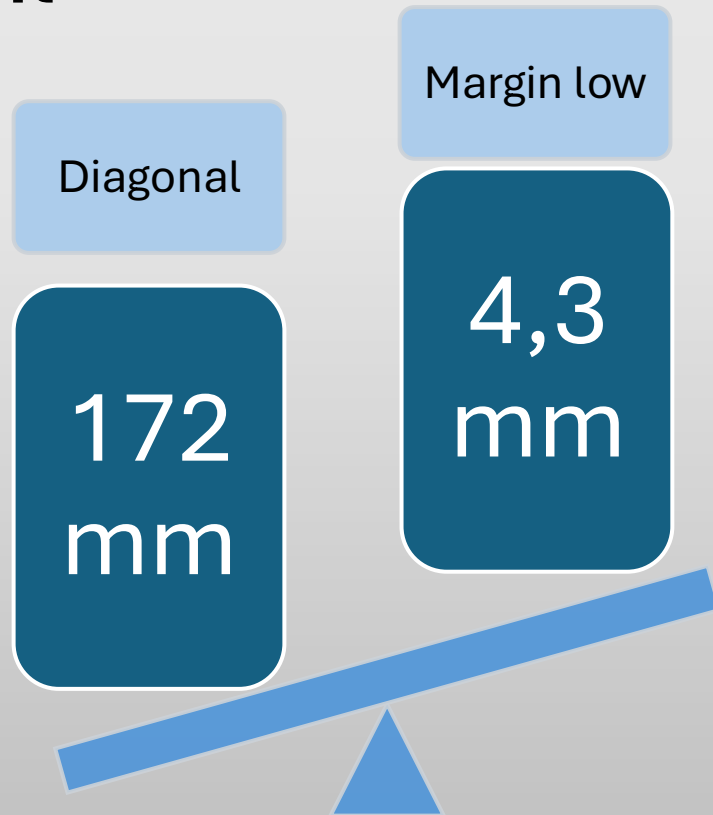


Calcul de la médiane

- Calcul fait après la séparation des données en jeu d'entraînement et jeu de test, seulement sur le jeu d'entraînement
- Éviter les fuites d'information du jeu de test vers mon modèle

La mise à l'échelle des données

Avant

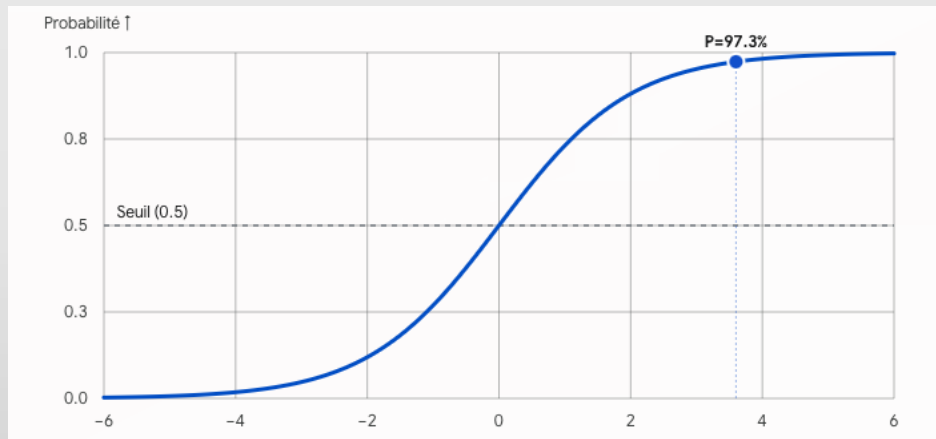


Après

Les données sont toutes sur la même échelle, centrées autour de zéro

→ Elles ont toutes le même poids dans l'analyse

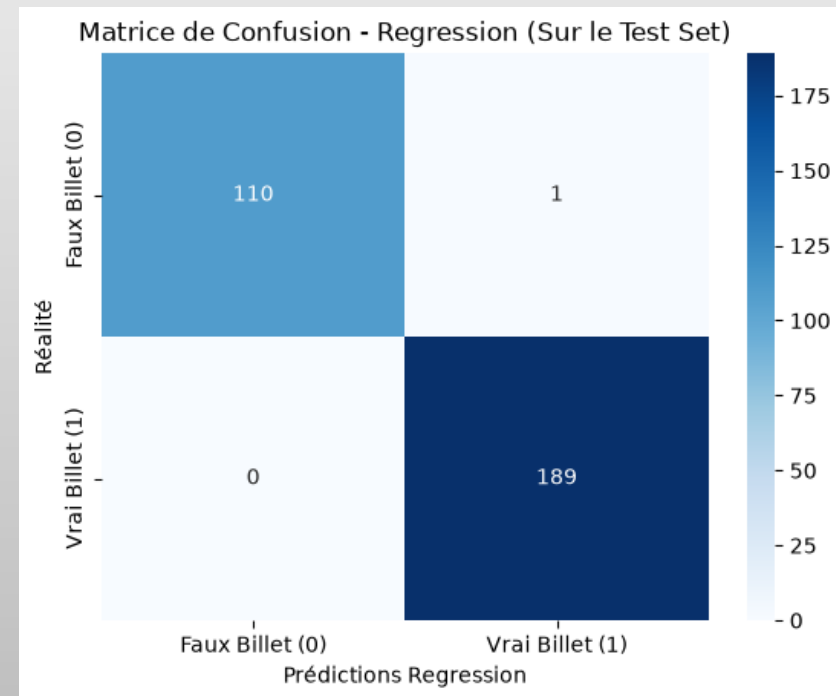
Régression logistique



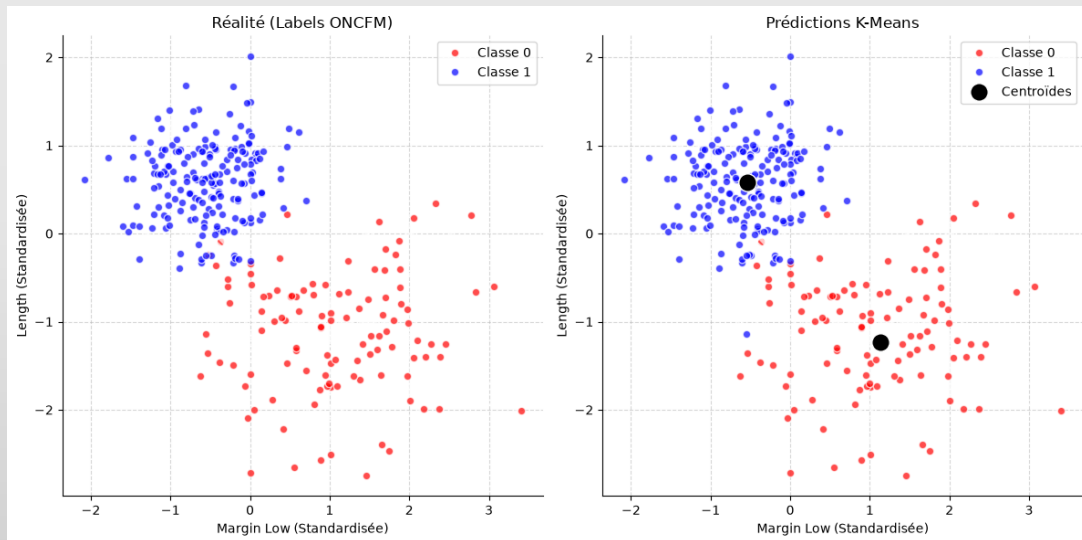
L'algorithme transforme les mensurations du billet en probabilité

Résultats

Score d'exactitude (accuracy) : 99,67 %



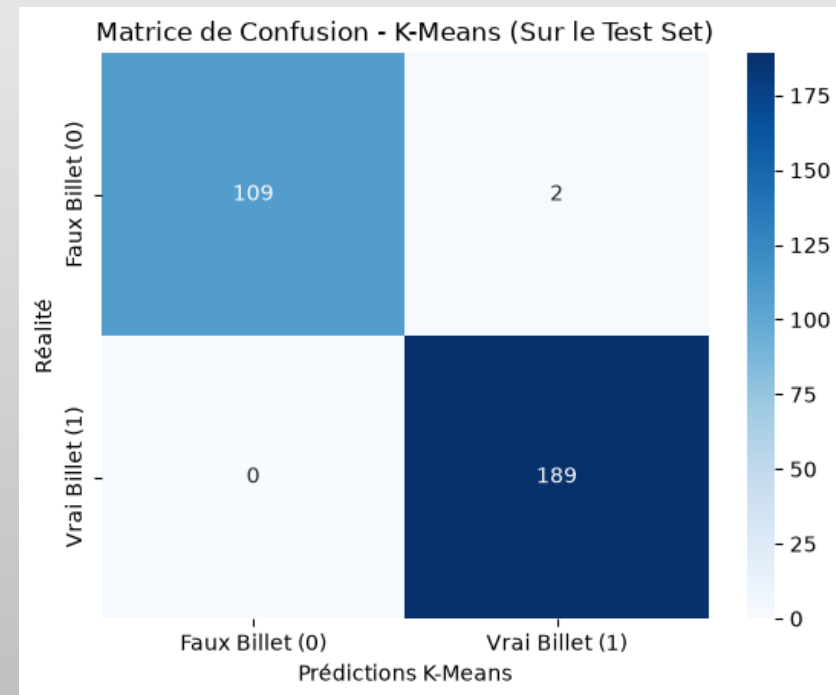
KMeans



L'algorithme calcule le centre de gravité des points, puis cherche celui le plus proche pour déterminer à quel groupe il appartient.

Résultats

Score d'exactitude (accuracy) : 99,33 %



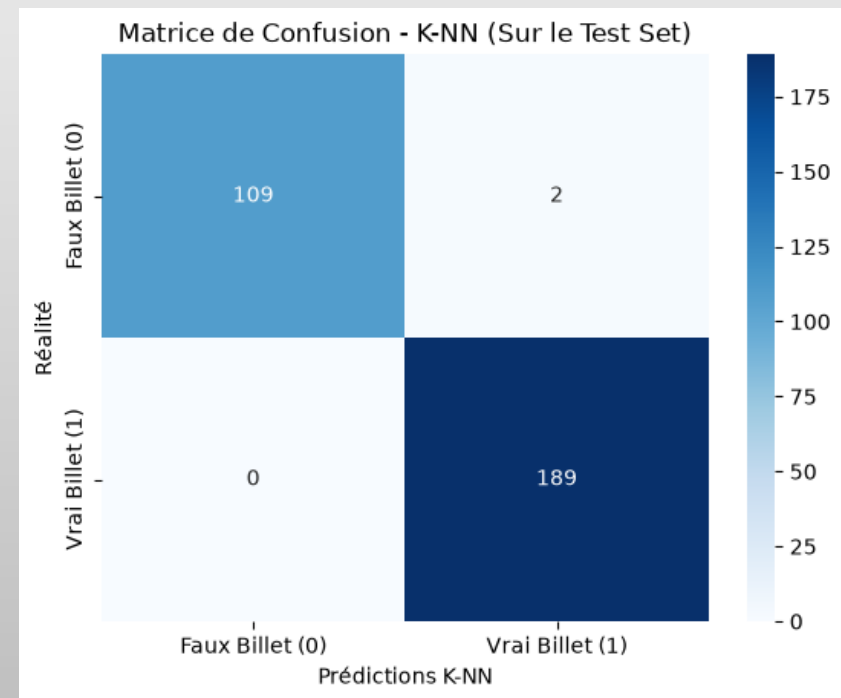
KNN (Key-Nearest Neighbor)



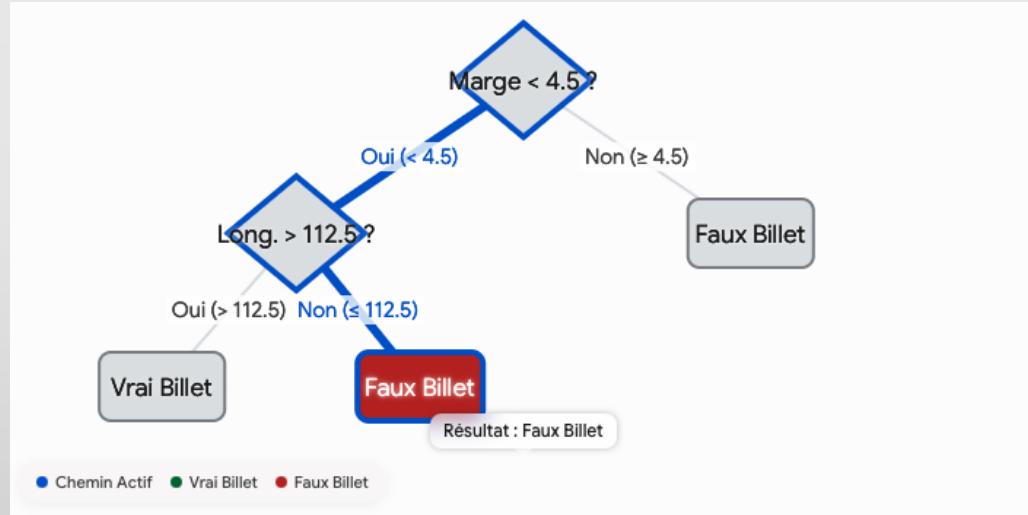
L'algorithme trouve les plus proches voisins, regarde s'ils sont vrais ou faux, et choisit la majorité.

Résultats

Score d'exactitude (accuracy) : 99,33 %



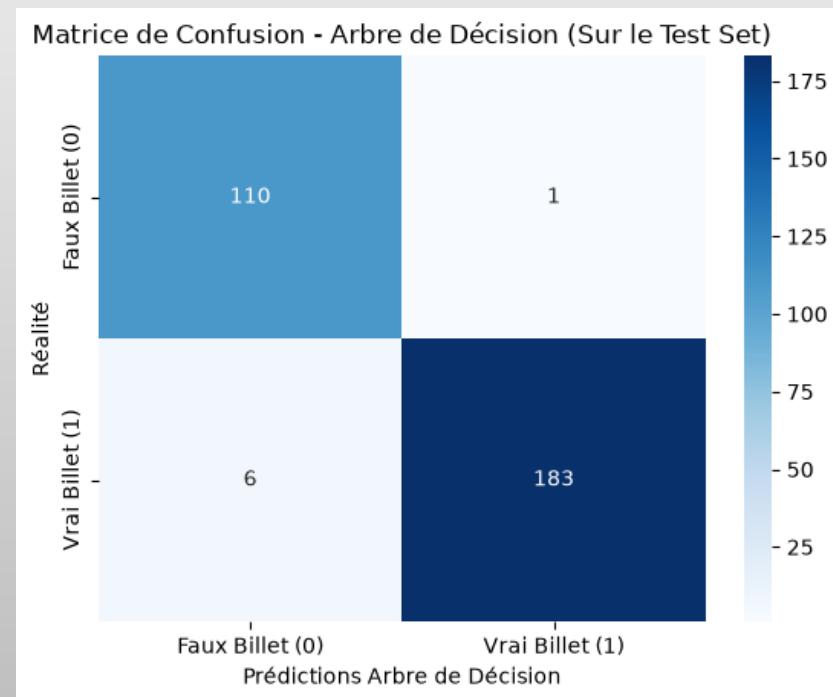
Arbre de décision



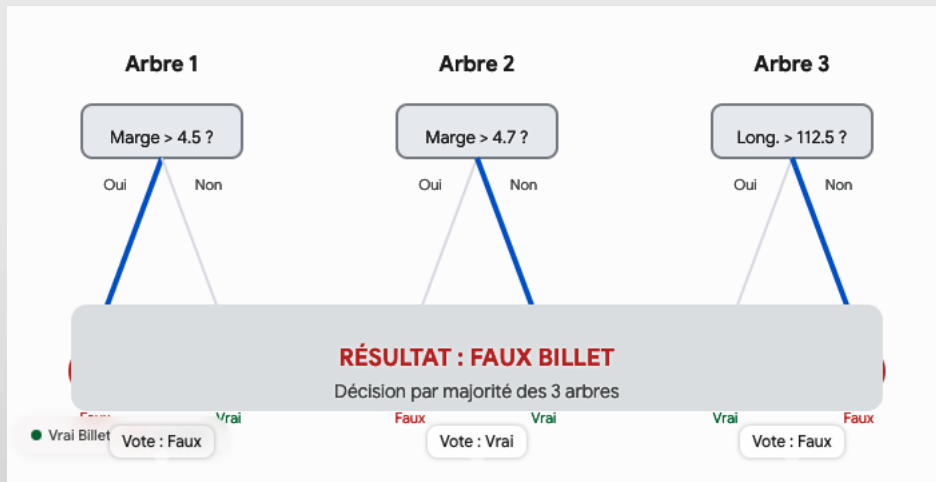
Une succession de questions simple permet de filtrer les vrais billets des faux.

Résultats

Score d'exactitude (accuracy) : 97,67 %



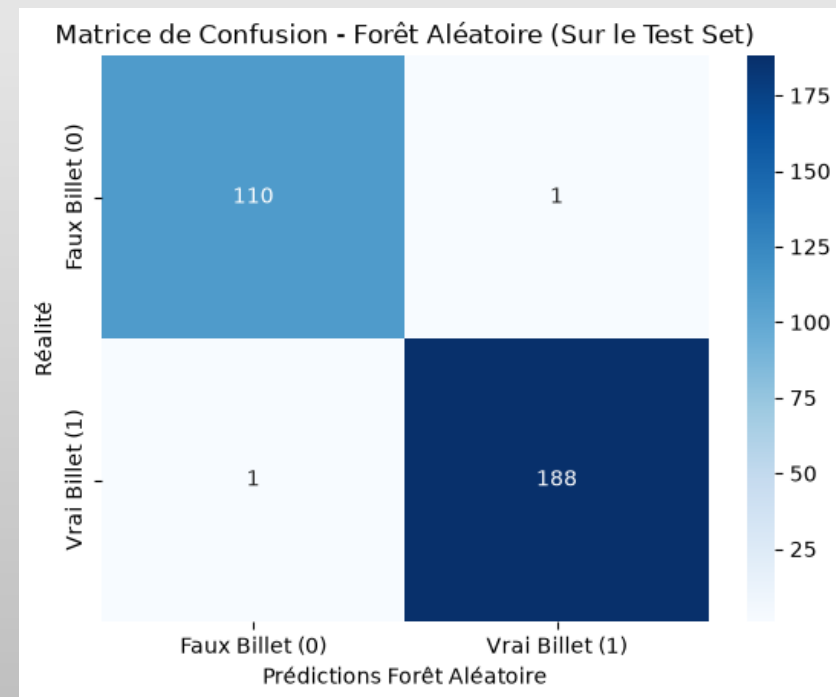
Forêt aléatoire



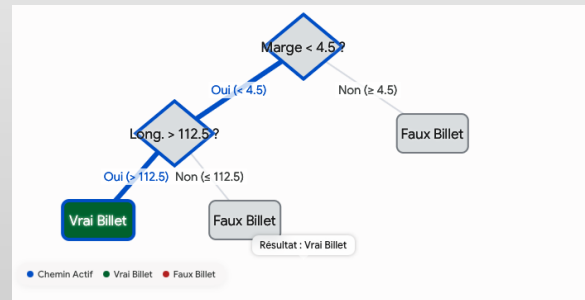
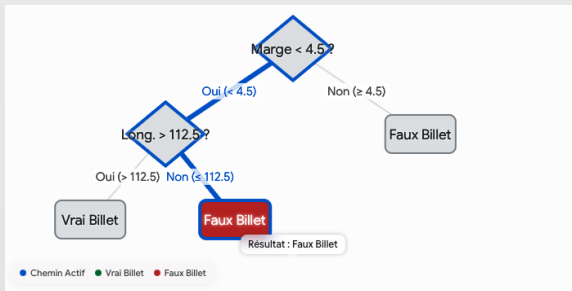
Plusieurs arbres de décision travaillent en même temps, et votent pour obtenir le résultat final

Résultats

Score d'exactitude (accuracy) : 99,33%



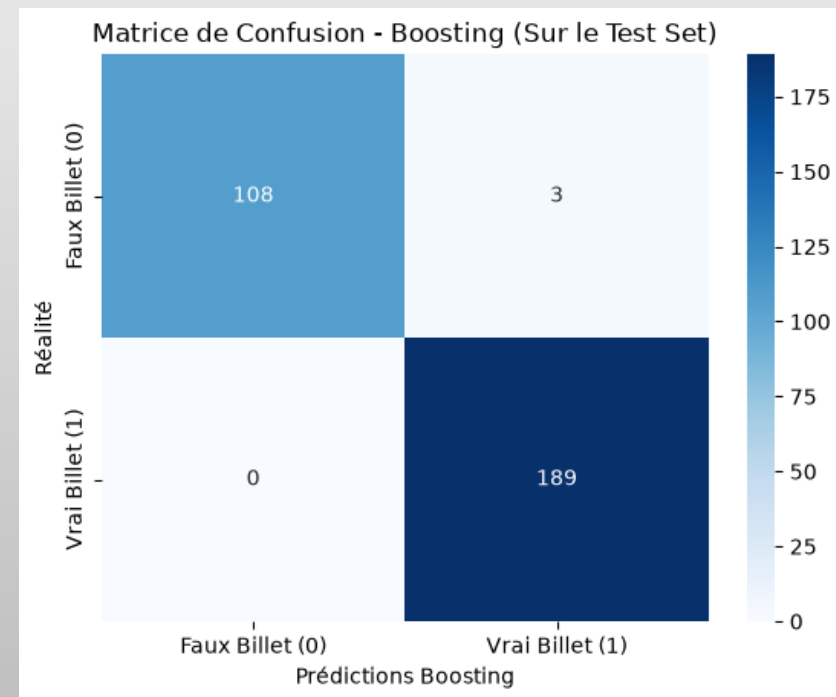
Gradient Boosting



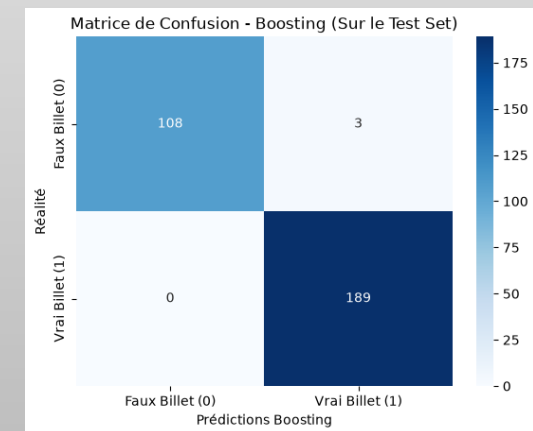
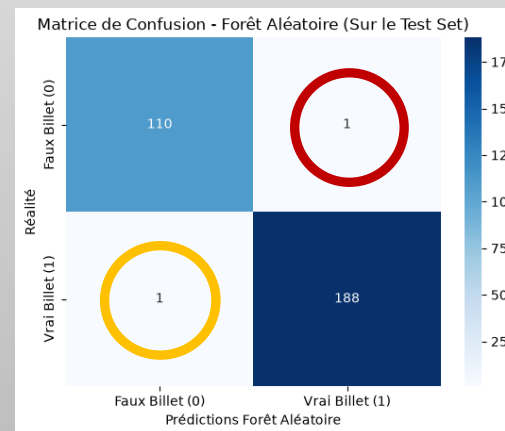
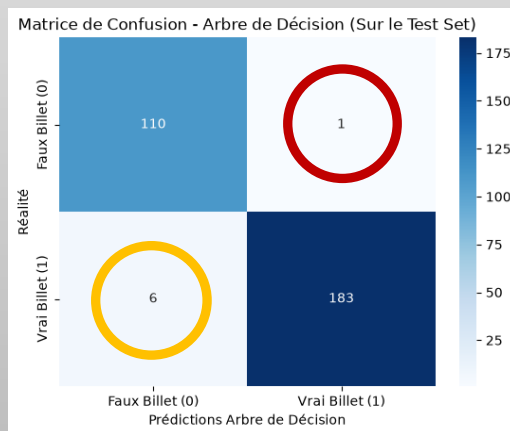
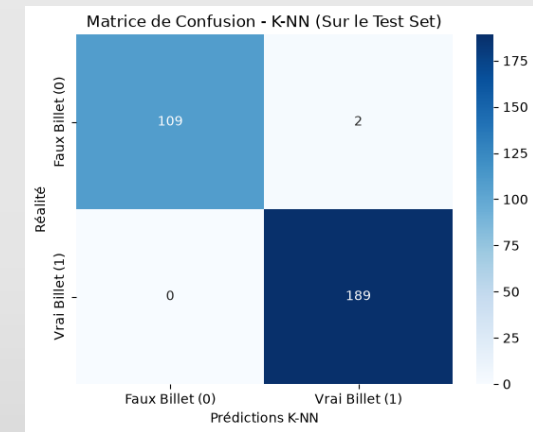
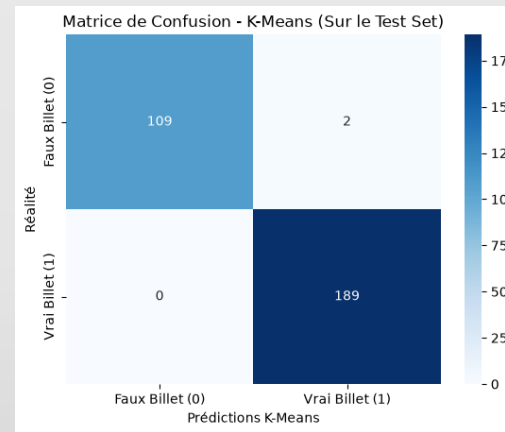
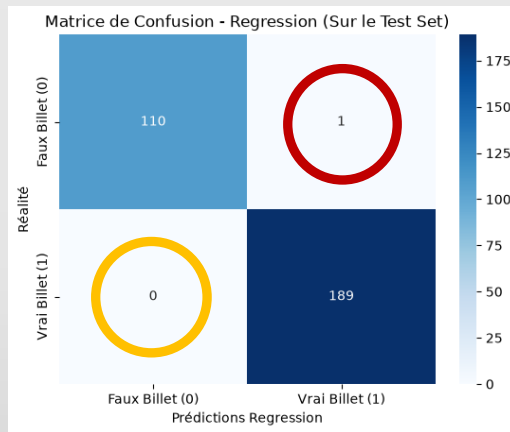
Les arbres de décision ne travaillent plus ensemble mais les uns après les autres, en se concentrant sur les erreurs pour s'améliorer.

Résultats

Score d'exactitude (accuracy) : 99%



Bilan des résultats



Choix final : la Régression Logistique

01

Sécurité financière :
risque de faux
positif minimisé

02

Transparence :
possibilité de
justifier plus
facilement les refus

03

Déploiement :
modèle plus léger et
rapide à exécuter

Conclusion

Démonstration du script

```
import joblib
import pandas as pd

def verifier_billets(chemin_fichier_csv):
    """
    Fonction qui prends en entrée un fichier csv comprenant 6 dimensions de billets.
    Retourne un fichier csv qui précise si les billets sont vrais ou faux.
    """

    model = joblib.load('billets_verification.pkl')
    scaler = joblib.load('scaler_oncfm.pkl')
    imputer = joblib.load('imputer_oncfm.pkl')

    df = pd.read_csv(chemin_fichier_csv)

    colonnes_mesures = ['diagonal', 'height_left', 'height_right', 'margin_low', 'margin_up', 'length']
    X_prod = df[colonnes_mesures].copy()

    X_prod_repare = imputer.transform(X_prod)
    X_prod_scaled = scaler.transform(X_prod_repare)

    predictions = model.predict(X_prod_scaled)
    probabilites = model.predict_proba(X_prod_scaled)[:, 1]

    df['probabilite_vrai'] = (probabilites * 100).round(2).astype(str) + ' %'
    df['resultat'] = ['Vrai' if p == 1 else 'Faux' for p in predictions]

    df.to_csv('resultats_analyse_billets.csv')
    print("Succès : les analyses ont été sauvegardés dans 'resultats_analyse_billets.csv'")

verifier_billets('data/billets_production.csv')
```